Neurocomputing 500 (2022) 1016–1028

Contents lists available at ScienceDirect

Neurocomputing

journal homepage: www.elsevier.com/locate/neucom

Functional gradient descent for *n*-tuple regression

Rafael F. Katopodis *, Priscila M.V. Lima, Felipe M.G. França

Universidade Federal do Rio de Janeiro (UFRJ), RJ, Brazil

ARTICLE INFO

Article history:
Received 7 February 2022
Revised 29 April 2022
Accepted 28 May 2022
Available online 1 June 2022
Communicated by Zidong Wang

Keywords:
Weightless neural networks
Kernel machines
Reinforcement learning
n-tuple regression

ABSTRACT

n-tuple neural networks have recently been applied to a wide range of learning domains. However, for the particular area of regression, existing systems have displayed two shortcomings: little flexibility in the objective function being optimized and an inability to handle nonstationarity in an online learning setting. A novel *n*-tuple system is proposed to address these issues. The new architecture leverages the idea of functional gradient descent, drawing inspiration from its use in kernel methods. Furthermore, its capabilities are showcased in reinforcement learning tasks, which involves both nonstationary online learning and task-specific objective functions.

1. Introduction

Weightless, or *n*-tuple, neural networks are general learning models that have been successfully employed in a variety of areas of learning, such as part-of-speech tagging [1], object tracking [2], biomarker classification [3], and open set recognition [4]. Based on the idea of memory elements as fundamental building blocks, these networks were originally proposed for pattern recognition tasks [5], but later extended into other domains, such as regression, in the form of the *n*-tuple regression network (NTRN) [6] and derived models [7].

The existing NTRN-based architectures have shown promise in multiple complex tasks, but face two shortcomings that limit their application. The first is that they are incapable of tracking nonstationary targets. The second is that they are fundamentally connected to a particular loss function and unable to optimize different objective functions. Both of these limitations prevent these models from being applied to problems involving online learning.

Several application domains, such as digital marketing or sensor networks for monitoring and control of plants, entail the generation of vast amounts of data. Given this volume and real-time nature of these applications, it is often the case that the data can only be properly handled sequentially, processing samples one at a time [8]. Furthermore, due to various sources of drift, such as seasonalities or equipment aging, the data might not admit being treated as samples from a common distribution [9]. These issues highlight the importance of developing learning algorithms suited for the nonstationary online learning scenario.

The structure of the present text is as follows: Section 2 presents the fundamental concepts of kernel machines and *n*-tuple neural networks, emphasizing the connection between them and the assumptions made by the *n*-tuple regression network that prevent it from being applicable to nonstationary online learning. Section 3 presents a new *n*-tuple model that leverages concepts from online learning with kernels to surpass the limitations listed. Section 4 showcases the model in tasks of policy search for reinforcement learning (RL), a domain that gives rise to both nonstationary online learning and task-dependent objective functions, and the paper is concluded in Section 5.

2. Fundamental concepts

2.1. Learning with kernels

Kernel methods are learning algorithms that primarily build upon the concept of similarity between inputs. Kernel models produce their responses for a given input by comparing it (either explicitly or implicitly) to training examples using a similarity measure, or a *kernel*. For the regression case, kernel models could take the form of (1), where $\{x_i \in \mathbb{R}^{d_{\text{in}}} | i = 1, \ldots, N\}$ is the set of N training samples, with d_{in} being the input dimensionality of the problem. $\{\alpha_i \in \mathbb{R}^{d_{\text{out}}} | i = 1, \ldots, N\}$ are the weight vectors associated to each training input, with d_{out} being the output dimensionality.

* Corresponding author.
E-mail address: rafaelkatopodis@gmail.com (R.F. Katopodis).

https://doi.org/10.1016/j.neucom.2022.05.114
0925-2312/

$$f(x) = \sum_{i=1}^{N} \alpha_i k(x_i, x) \tag{1}$$

An alternative view of how a model like this works is that an output is made up of contributions of all samples, and how strongly a sample contributes is a function of how similar it is to the query input, according to the kernel in use. Kernel methods propose a supervised learning paradigm where training samples have an active role when the model is put to use. This could be contrasted to parametric models, where the data set is used only insofar as finding an appropriate setting of parameters.

A lot relies on how good a similarity measure the kernel is in building models in the form (1) that are useful. The appropriateness of a measure, however, is not an intrinsic quality, but a task-dependent one. For a linearly separable classification problem in a Euclidean space, for instance, the Euclidean distance could be a perfectly reasonable similarity metric. For more elaborate contexts, on the other hand, such as image classification, simple measures probably won't do. In these more complex scenarios, however, it may be far from obvious how to design contextually relevant similarity functions. One idea that comes up in cases such as these is to map inputs into a more suitable, possibly higher dimensional, space, one where simple, geometric-based, measures became appropriate. This is a central idea for kernel-based algorithms, and kernels able to measure similarity in higher dimensional spaces in a computationally efficient manner are of special interest.

2.1.1. Feature maps and positive-definite kernels

The kernels that are generally adopted in kernel methods are ones that can be stated as inner products in feature spaces (2), where ϕ is a *feature map*, a function that maps inputs into the desired feature space.

$$k(w, x) = \langle \phi(w), \phi(x) \rangle \tag{2}$$

Furthermore, the kernels sought are ones able to perform these inner products without having to *explicitly* map into feature spaces. The reason being that first mapping inputs into a high-dimensional space and subsequently computing the inner product of these large vectors could be rather costly, both in terms of memory and time complexity. For instance, with $w, x \in \mathbb{R}^m$, the polynomial kernel $k_{\text{poly}}(w, x) = \langle w, x \rangle^d$ requires only an inner product in the input space and an exponentiation operation. Its corresponding feature map, however, consists of all dth degree products of entries of an input vector, yielding a feature vector with $\binom{d+m-1}{d}$ dimensions [10]. If $m = 3$ and $d = 10$, this would imply 66-dimensional feature vectors.

Kernels that admit being stated as above are also known as positive-definite (PD) kernels, and this is an if and only if condition, meaning that if a map ϕ can be found such that (2) holds, the kernel must be PD [11]. A PD kernel can have more than one associated feature space. One special feature space that can always be associate to a PD kernel k by construction is one where the members are itself functions. The feature map for this space is given by $\phi(x)(\cdot) = k(\cdot, x)$. That is, x is mapped to $k(\cdot, x)$, the function that compares any valid input to x.

This function space can be used to highlight some important properties of PD kernels. First, the feature space is a *vector* space, meaning that members can be added together and multiplied by scalars to produce new members. $f(\cdot)$ and $g(\cdot)$ in (3), for instance, are valid members of the feature space. These linear combinations are of the same form as how kernel models where presented, in (1).

$$f(\cdot) = \sum_{i=1}^{m} k(\cdot, x_i), \quad g(\cdot) = \sum_{j=1}^{m'} \beta_j k(\cdot, x'_j) \tag{3}$$

Therefore, a learned model is one particular member of the function space spanned by its kernel. An inner product can be defined in the space (4), and the operation satisfies linearity (5).

$$\langle f, g \rangle = \sum_{i=1}^{m} \sum_{j=1}^{m'} \alpha_i \beta_j k(x_i, x'_j) \tag{4}$$

$$\langle \lambda f, g \rangle = \lambda \langle f, g \rangle \tag{5}$$

Finally, (6) follows directly from the definition of inner product and vector spaces. This is known as the *reproducing property*, and has the special case (7).

$$\langle k(\cdot, x), f \rangle = f(x) \tag{6}$$

$$\langle k(\cdot, x), k(\cdot, x') \rangle = k(x, x') \tag{7}$$

2.1.2. Online learning

The training of kernel models usually takes place in an off-line manner, and assumes all data are known as a large batch before any learning can take place, such as in the popular Support Vector algorithms [11]. However, domains such as reinforcement learning present a problem setting where data becomes available only gradually. Batch algorithms could be adapted to this context by using a sliding window over the data, but that would constitute a computationally-intensive and wasteful recourse, as models would be trained from scratch and then discarded. It is possible, though, to learn kernel models in a manner more suitable to the online setting by leveraging the idea of gradient-based optimization.

Gradient descent is a widely used technique in the context of parametric models to navigate in the space of possible parameters, in search of a locally optimal setting. In the context of kernel models, on the other hand, the gradient is to be used as a tool to navigate in the function space spanned by the chosen kernel. The interpretation that allows this usage for learning with kernels is of the gradient as the linear component of a change in a function in face of a small change ϵ in its input [12]. For functions of real or vector values, this could be seen as (8). However, in the function space-view of the problem, the interest is in taking gradients of *functionals*: functions that take as input other functions. The concept of gradient following this interpretation can be extended to the functional case through the implicit definition in (9), where E is a functional.

$$h(x + \epsilon) = h(x) + \epsilon \nabla h(x) + \mathcal{O}(\epsilon^2) \tag{8}$$

$$E[f + \epsilon g] = E[f] + \epsilon \langle \nabla_f E[f], g \rangle + \mathcal{O}(\epsilon^2) \tag{9}$$

In a similar way to traditional gradient descent, the functional version makes updates seeking to minimize a loss functional. Let f denote the model. At time step t, an update is made upon observing a new sample by taking a step in the opposite direction of the gradient, with step size η_t, according to the update rule (10). $L_{x,y}$ is the single-sample loss functional.

$$f_{t+1} \leftarrow f_t - \eta_t \nabla_{f_t} L_{x_t, y_t}[f_t] \tag{10}$$

The key element in computing the gradient of a loss functional is first writing it in terms of the *evaluation functional* (11), which, given a function, returns its value when evaluated at a given input. The gradient of the evaluation functional can be derived following the implicit definition (9) and the reproducing property (6): $\nabla_f E_x[f] = k(x, \cdot)$ (12).

$$E_x[f] = f(x) \tag{11}$$

$$\begin{aligned} E_x[f + \epsilon g] &= f(x) + \epsilon g(x) + 0 \\ E_x[f + \epsilon g] &= f(x) + \epsilon \langle k(x, \cdot), g \rangle + 0 \\ E_x[f + \epsilon g] &= f(x) + \epsilon \langle \nabla_f E_x, g \rangle + \mathcal{O}(\epsilon^2) \end{aligned} \tag{12}$$

For instance, if the chosen loss is the squared error functional (13), its gradient can be computed by applying the chain rule and replacing in the gradient of the evaluation functional (14). This example highlights that functional gradients are itself functions, and their update steps add a new function to the kernel expansion of the model, essentially moving it in the function space associated to the PD kernel in use. Once again, this can be contrasted to gradient descent in parametric models, where steps update a fixed set of parameters.

$$\begin{aligned} L_{x,y}[f] &= (y - f(x))^2 \\ &= (y - E_x[f])^2 \end{aligned} \tag{13}$$

$$\begin{aligned} \nabla_f L_{x,y}[f] &= \nabla_f (y - E_x[f])^2 \\ &= -2(y - E_x[f])^2 \nabla_f E_x[f] \\ &= -2(y - f(x))^2 k(x, \cdot) \end{aligned} \tag{14}$$

One issue in the functional approach, therefore, is that the kernel expansion grows with the number of samples, which would result in growing memory usage and inference-time complexity if the member functions in the expansion were all to be stored in a list. This functional gradient-based approach for online learning with kernels have been used in *NORMA* algorithms [13], which additionally propose dealing with growing complexities by maintaining only a fixed number of the most relevant member functions. Furthermore, functional gradient descent has been employed in the past in the context of policy search [14].

2.2. n-tuple neural networks

Weightless, or n-tuple, neural networks are a class of learning architectures very loosely inspired by the workings of the brain, with a long history [15,5]. In contrast to the McCulloch-Pitts neuron [16], where its output is a function of an inner product between the neuron's inputs and a weight vector, a RAM neuron consists of a memory element, and uses its input to address a position. The content of an addressed position is what a RAM neuron outputs for a given input. In order to realize this addressing, it is expected that the input is a binary pattern. This needs not be a hard limitation on the domains RAM neurons and weightless nets can be applied, as techniques can be applied to encode other forms of inputs into suitable binary patterns.

One of the simplest RAM neurons is one where a memory position stores only a single bit (Fig. 1a). Learning with such a neuron is quite simple. Given a desired input–output mapping, it can be learned by setting the bit addressed by the input to the value of the output, without affecting any other positions. Thus, this kind of neuron is able to learn any Boolean mapping. A big downside, however, is that RAM neurons cannot generalize. What was learned from a given input cannot benefit some other one, no matter how similar the two are, because they address distinct positions and have, therefore, uncorrelated outputs. One of the essential properties of a learning model is that of being able to generalize to previously unseen entries. A single RAM neuron is incapable of doing that. However, generalization can be obtained by organizing these neurons into networks [17,18].

2.2.1. Discriminator networks

One of the simplest network organizations able to exhibit a generalization behavior is the *discriminator* (Fig. 1b). It consists of a set of single-bit RAM neurons, and the way that it approaches generalization is having each of these neurons observe only part of the binary input pattern. The subset of the input under observation by a particular neuron is commonly referred to as a *tuple*. The alternative name for models such as these, n-tuple networks, comes from this convention, with n denoting the number of bits that make up a tuple. In this work, tuples are formed by randomly sampling the bits from the input pattern, with no overlap between tuples. The network's output, known as the discriminator's *response*, is computed as the sum of the activations of the neurons. Therefore, a discriminator is able to produce for a novel input a response that extrapolates from the training data, as long as at least one neuron is able to match its tuple to the ones seen during training.

The discriminator architecture can serve as the basis for several more complex n-tuple models. Its response ranges from 0 up to the number of neurons, N. A response of zero occurs when all tuples in the input are completely foreign to their neurons. N, on the other hand, happens when the input is the same as one seen during training. Intermediate values of response denote a partial level of recognition. In this way, discriminators can be regarded as learned similarity functions, comparing their inputs to the concept underlying the training samples. This similarity function could be used as a building block for a classification system, with several discriminators, each in charge of recognizing one class among all classes under consideration (Fig. 2a). To classify a novel input, the pattern is presented to each discriminator, and the one with the greatest response attributes its class to it. Multidiscriminator classifier systems such as these are known as *WiSARD* models, in reference to one of the first implementations [19].

2.2.2. n-tuple regression

The workings of the RAM discriminator lend it almost immediately to use in classification tasks. However, it can be adapted to the domain of regression with only few changes. The n-tuple regression network (NTRN) [6] does precisely so, serving as a model capable of approximating real-valued functions. The network is organized as the RAM discriminator seen before, but its neurons differ. Instead of being single-bit, memory positions store two values: a real value estimate, V, and a counter, C. When training on a data set $\{(\mathbf{x}_t, y_t)\}_{t=1}^{T}$, consisting of pairs of inputs and target values, the network is updated, for each pair, by simply adding the target to the addressed estimates, and incrementing the addressed counters, as in (15). Here, $V^{[i]}(\mathbf{x}_t)$ denotes the estimate addressed by input $\mathbf{x}_t$ in neuron i, and $C^{[i]}(\mathbf{x}_t)$, the counter addressed by it in the same neuron. The output of the model is computed by summing the addressed estimates and dividing by the addressed counters (16). The network is depicted in Fig. 2b.

$$\begin{cases} V^{[i]}(\mathbf{x}_t) \leftarrow V^{[i]}(\mathbf{x}_t) + y_t \\ C^{[i]}(\mathbf{x}_t) \leftarrow C^{[i]}(\mathbf{x}_t) + 1 \end{cases}, \; i = 1, \ldots, N, \; t = 1, \ldots, T \tag{15}$$

$$f(\mathbf{x}) = \frac{\sum_{i=1}^{N} V^{[i]}(\mathbf{x})}{\sum_{i=1}^{N} C^{[i]}(\mathbf{x})} \tag{16}$$

The NTRN highlights the connection that can be drawn between n-tuple nets and kernel methods, as its architecture, update rule and output function were designed to implement, in a memory and computationally efficient manner, a Nadaraya-Watson kernel regression [20,21]. This connection can be made clear by first recognizing that the tuple sampling performed by networks such as the NTRN naturally lead to a distance measure between inputs. Let $\tau^{[i]}(\mathbf{x})$ be the tuple observed by neuron i for input $\mathbf{x}$. That is, $\tau^{[i]}(\mathbf{x})$ denotes the address that $\mathbf{x}$ indexes in neuron i. A distance measure can be defined for two patterns in terms of the number of differing tuples between them (17).

$$\rho(\mathbf{x}, \mathbf{z}) = \sum_{i=1}^{N} [\![\tau^{[i]}(\mathbf{x}) \neq \tau^{[i]}(\mathbf{z})]\!] \tag{17}$$

If two patterns are identical, all their tuples are the same, and, as would be expected, their tuple distance is zero. On the other hand, if the patterns differ, some of their tuples wouldn't be the

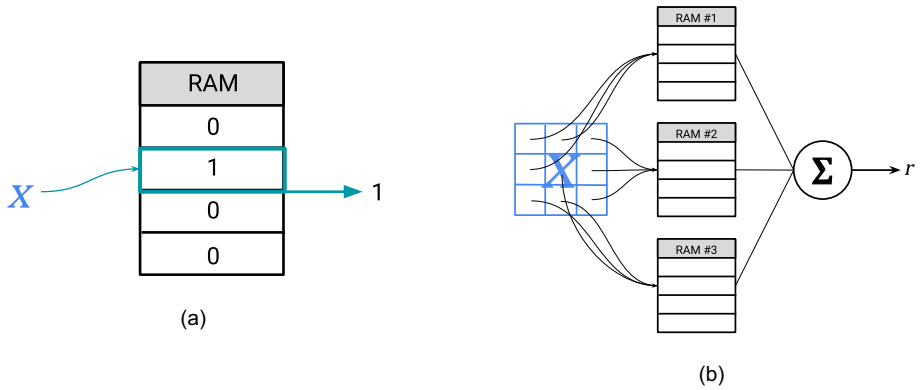


Fig. 1. (a) Single-bit RAM neuron. Its activation consists simply of the state of the bit addressed by the input. (b) RAM discriminator, a network of RAM neurons that behaves as a learned similarity function. Each neuron observes only part of the input, in a manner that the network is able to generalize to novel patterns.

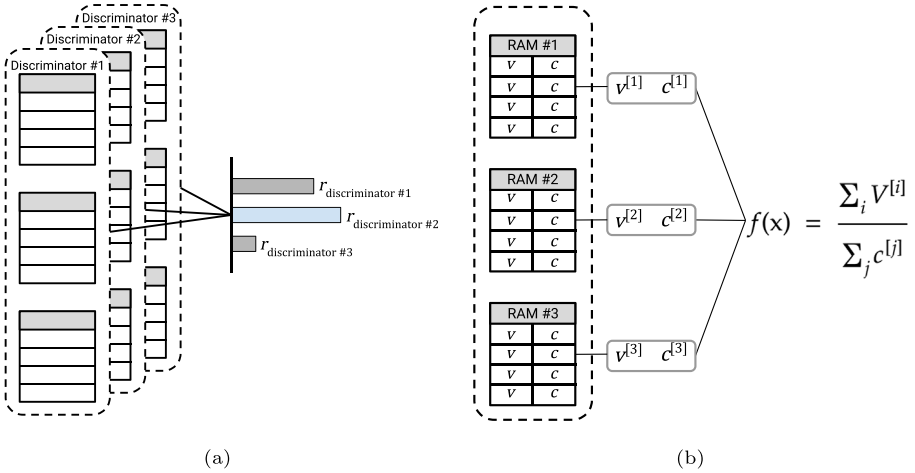


Fig. 2. (a) A multidiscriminator classifier, also known as a WiSARD model. Each discriminator is trained with samples of a particular class. When the model is employed to classify an input, all discriminators provide a response to it, and a class is assigned based on the greatest response. (b) A *n*-tuple regression network, an *n*-tuple net made up of RAM neurons that hold a real value estimate and a counter per memory position. The update rule and output function of the model are given by (15) and (16).

same, resulting in a strictly positive distance. The tuple function τ can further be used in defining a indicator variable $M_t^{[i]}(\mathbf{x})$, denoting whether the pattern $\mathbf{x}$ addresses the same position as sample $\mathbf{x}_t$ in neuron i (18).

$$M_t^{[i]}(\mathbf{x}) = \begin{cases} 1 & \text{if } \tau^{[i]}(\mathbf{x}) = \tau^{[i]}(\mathbf{x}_t) \\ 0 & \text{otherwise} \end{cases} \tag{18}$$

At first nonintuitive, this indicator allows expressing both the value and the counter addressed in a neuron, $V^{[i]}(\mathbf{x})$ and $C^{[i]}(\mathbf{x})$, in terms of a summation over the entire data set ((19), (20)). Furthermore, the indicator can be related to the tuple distance when summed over all neurons (21).

$$V^{[i]}(\mathbf{x}) = \sum_{t=1}^{T} y_t M_t^{[i]}(\mathbf{x}) \tag{19}$$

$$C^{[i]}(\mathbf{x}) = \sum_{t=1}^{T} M_t^{[i]}(\mathbf{x}) \tag{20}$$

$$\sum_{i=1}^{N} M_t^{[i]}(\mathbf{x}) = N - \rho(\mathbf{x}, \mathbf{x}_t) = N\left(1 - \frac{\rho(\mathbf{x}, \mathbf{x}_t)}{N}\right) \tag{21}$$

With (20) and (21), it is possible to demonstrate the equivalence between the NTRN and a Nadaraya-Watson estimator (22). The proof also stresses the kernel function being implicitly used by the network under this perspective.

$$\begin{aligned} f(x) &= \frac{\sum_{i=1}^{N} V^{[i]}(\mathbf{x})}{\sum_{i=1}^{N} C^{[i]}(\mathbf{x})} \\ &= \frac{\sum_{i=1}^{N}\sum_{t=1}^{T} y_t M_t^{[i]}(\mathbf{x})}{\sum_{i=1}^{N}\sum_{t=1}^{T} M_t^{[i]}(\mathbf{x})} && \text{Using (19) and (20)} \\ &= \frac{\sum_{t=1}^{T} y_t \sum_{i=1}^{N} M_t^{[i]}(\mathbf{x})}{\sum_{t=1}^{T}\sum_{i=1}^{N} M_t^{[i]}(\mathbf{x})} \\ &= \frac{\sum_{t=1}^{T} y_t N\left(1-\frac{\rho(\mathbf{x},\mathbf{x}_t)}{N}\right)}{\sum_{t=1}^{T} N\left(1-\frac{\rho(\mathbf{x},\mathbf{x}_t)}{N}\right)} && \text{Using (21)} \\ &= \frac{\sum_{t=1}^{T} y_t \left(1-\frac{\rho(\mathbf{x},\mathbf{x}_t)}{N}\right)}{\sum_{t=1}^{T} \left(1-\frac{\rho(\mathbf{x},\mathbf{x}_t)}{N}\right)} \\ &= \frac{\sum_{t=1}^{T} y_t k(\mathbf{x},\mathbf{x}_t)}{\sum_{t=1}^{T} k(\mathbf{x},\mathbf{x}_t)}, \\ & k(\cdot,\cdot) = \left(1 - \frac{\rho(\cdot,\cdot)}{N}\right) \end{aligned} \tag{22}$$

Given the equivalence in behavior, the appeal of using a NTRN instead of an explicit Nadaraya-Watson kernel estimator is that the weightless network allows inferring outputs in constant time and uses a constant amount of memory, once the number of neurons and tuple size has been set. An explicit kernel machine, on the other hand, would have computational and storage requirements proportional to the number of samples.

The implementation of a kernel estimator is what endows this architecture with the capability of performing regression, but it is also the source of two limitations. The Nadaraya-Watson estimator is an approximation of the conditional expectation (23), which is, in turn, the estimator that minimizes the mean squared error (24). Therefore, this family of n-tuple networks is coupled with the mean squared error as its objective function, which limits the problem domains in which they can be applied.

$$\frac{\sum_{t=1}^{T} y_t k(\mathbf{x}, \mathbf{x}_t)}{\sum_{t=1}^{T} k(\mathbf{x}, \mathbf{x}_t)} \approx \frac{\int y \cdot p(\mathbf{x}, y) \mathrm{d}y}{p(x)} \tag{23}$$

$$\begin{aligned} &= \mathbb{E}[Y|\mathbf{X} = \mathbf{x}] \\ &\in \operatorname{argmin}_g \mathbb{E}[(Y - g(\mathbf{x}))^2 | \mathbf{X} = \mathbf{x}] \\ &= \operatorname{argmin}_g \mathrm{Var}[Y - g(\mathbf{x}) | \mathbf{X} = \mathbf{x}] + \mathbb{E}[Y - g(\mathbf{x}) | \mathbf{X} = \mathbf{x}]^2 \\ &= \operatorname{argmin}_g \mathrm{Var}[Y|\mathbf{X} = \mathbf{x}] + \{\mathbb{E}[Y|\mathbf{X} = \mathbf{x}] - g(\mathbf{x})\}^2 \end{aligned} \tag{24}$$

The second shortcoming of this learning model is the underlying assumption that the joint distribution of input and target variables, $p_{\mathbf{X},Y}(\mathbf{x}, y)$, is fixed, albeit unknown. Such premise can be reasonable in settings where all samples are available before any training takes place and a deterministic mapping must be uncovered from it. In a context of where data presents itself gradually, such as online or reinforcement learning, however, the joint distribution may change over time. Nevertheless, the NTRN always attributes an equal importance to all samples, precluding it from properly handling nonstationary processes. Fig. 3 illustrates how an estimate can deteriorate if it attributes equal importance to samples that are drawn from a distribution that shifts over time.

2.2.3. Encoding schemes

As mentioned previously, the formation of tuples and addressing of RAM neurons require inputs to be binary patterns. To handle data in different formats, a preprocessing step is necessary. However, not any conversion scheme is appropriate. n-Tuple networks measure the similarity between binary inputs in terms of the tuple distance (17), which could be regarded as a generalized Hamming distance. Therefore, in order to preserve the similarity between inputs, an encoding scheme should map similar inputs to binary patterns with a small tuple distance between them. For instance, two IEEE 754-encoded numbers with small tuple distance may not necessarily represent close numbers, as that will depend on where on the pattern the differing bits lie (sign, exponent or significand parts). In many learning tasks, inputs are vectors of real values. In the present work, two encoding schemes are considered in order to binarize samples of this nature: the thermometer [22] and circular encoders [23]. These are suitable for encoding scalars. To binarize vectors, it only takes encoding its components individually and stacking the resulting patterns together.

2.2.4. Thermometer encoding

The thermometer encoder takes its name from the resemblance of its behavior to the one of a thermometer when submitted to different temperatures. Under this scheme, real values are mapped to binary arrays that are more or less filled depending on whether it's closer to a given minimum or maximum. Formally, let $\mathbf{x}$ be the value to be encoded, defined in a interval $[l, u]$, and S, the output vector, made up of R bits. S_i is the value of the i-th bit of S, given by Eq. 25.

$$S_i = \begin{cases} 1 & \text{if } \mathbf{x} > l + (i-1) \times \frac{u-l}{R}, \\ 0 & \text{otherwise} \end{cases} \quad i \in [1, R] \tag{25}$$

The thermometer encoding splits the interval in which the input is defined into R subintervals, and all values belonging to a given subinterval are mapped to a identical binary pattern. The extent of this aliasing of inputs is regulated by the encoding parameter R. On one hand, the effect may be detrimental, as the network becomes unable to tell apart aliased inputs. On the other hand, it enables the model to draw answers to previously unseen inputs, favoring generalization.

2.2.5. Circular encoding

\The second encoding scheme considered, circular encoding, maps to patterns with a fixed number of activated bits, arranged contiguously as a block. Larger inputs, in terms of how close they are to the upper bound of their interval, are mapped to patterns where the block of activated bits is more shifted to the right. Furthermore, the activated bits can be shifted beyond the edge of the pattern, wrapping around. Besides aliasing inputs that fall into the same subinterval, this circular behavior favors a radial similarity, as extreme values are mapped to similar patterns.

Instead of starting from a equation that determines the state of given bit in the encoded pattern, as with the thermometer encoding, the circular encoding of an input is more simply described as a two-step procedure. First, start with a base pattern of R bits, where the first $\lfloor R/2 \rfloor$ ones are activated. Next, right-shift the pattern n_{shift} units, given by (26), where, once again, $[l, u]$ is the interval where $\mathbf{x}$ is defined and R, the encoding resolution. Fig. 4 presents examples of both thermometer and circular encodings.

$$n_{\text{shift}} = \left\lfloor \frac{\mathbf{x} - l}{u - l} \times R \right\rfloor \tag{26}$$

3. The functional gradient-inspired n-tuple network

In order to tackle the shortcomings of the NTRN listed in Section 2.2.2, a new architecture is proposed, organized as a RAM discriminator. Unlike the NTRN, however, the memory positions do not need to store access counters. They hold only values, that are either scalars or vectors, depending on the dimensionality of the target function. One additional difference between the networks is that the new model requires the choice of an error metric, not having one implicitly.

The update rule and the output function are given by (27), (28) and (29). The notation is similar to the one used by the NTRN. $(\mathbf{x}_t, y_t)$ is the training sample at time t, consisting of an input and a target. $v_t^{[i]}(\mathbf{x})$ denotes the value addressed by the tuple formed by $\mathbf{x}$ for neuron i after t samples. Once again, $v_0^{[i]}(\mathbf{x}) = 0, \forall \mathbf{x} \in \mathscr{X}, i = 1, \ldots, N$. $c : \mathscr{X} \times \mathbb{R}^2 \mapsto \mathbb{R}$ is a differentiable loss function, taking an input, its correct target and a target prediction and returning a measure of the error made by the prediction. Finally, η_t is a learning rate that regulates how much values change in the direction of the gradient.

$$v_t^{[i]}(\mathbf{x}_t) = v_{t-1}^{[i]}(\mathbf{x}_t) + \eta_t \delta_t, \qquad i = 1, \ldots, N \tag{27}$$

$$\delta_t = \partial_{f_{t-1}(\mathbf{x}_t)} c(\mathbf{x}_t, y_t, f_{t-1}(\mathbf{x}_t)) = \frac{\partial}{\partial z} c(\mathbf{x}_t, y_t, z)|_{f_{t-1}(\mathbf{x}_t)} \tag{28}$$

$$f_t(\mathbf{x}) = \frac{1}{N} \sum_{i=1}^{N} v_t^{[i]}(x) \tag{29}$$

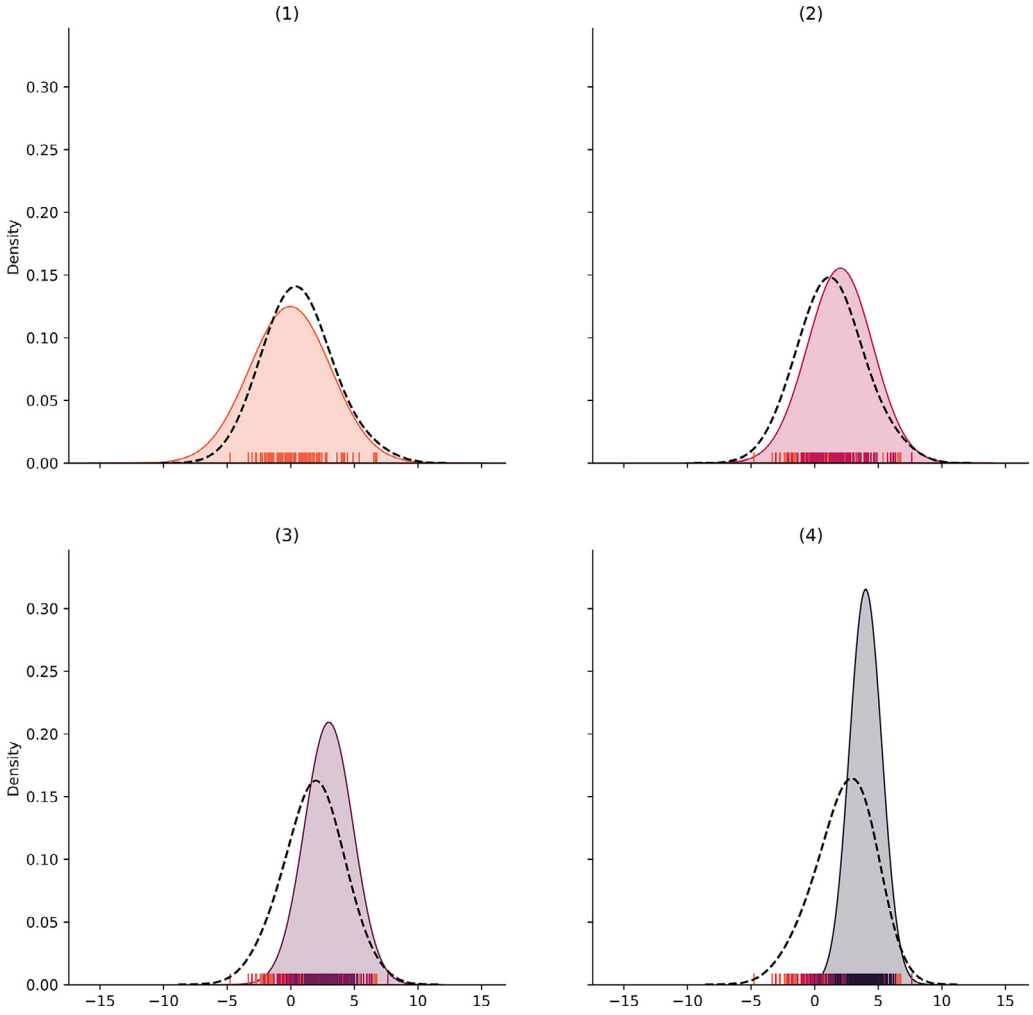


Fig. 3. A simplified example of the negative effect of an estimation that assumes stationarity in a nonstationary environment. The solid line denotes the probability density function from the true underlying distribution,ticks along the *x* axis denote samples drawn from it, and the dashed line denotes a density estimate from all available samples. As the distribution shifts, the estimate becomes a progressively worst approximation of the true density.

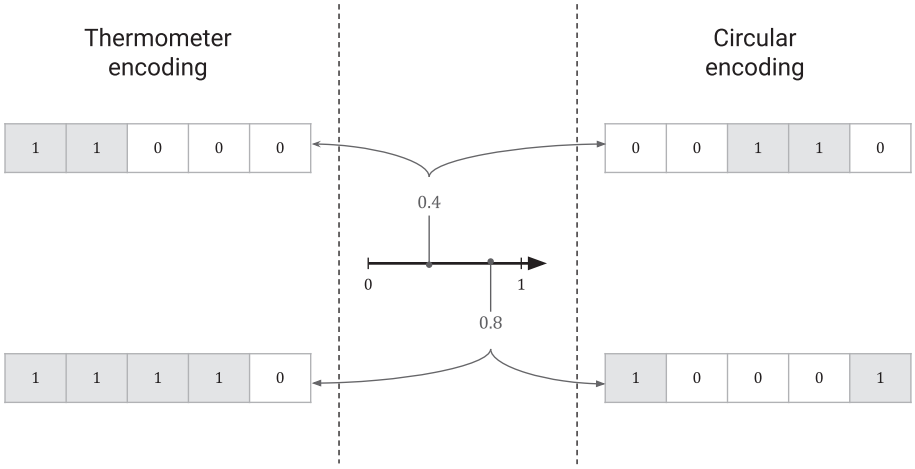


Fig. 4. Thermometer and circular encodings with 5-bit resolutions of two real values defined in the $[0, 1]$ interval. While thermometer encodings entail a variable number of active bits, circular encodings have a fixed set given a resolution. The example also showcases the wrap-around behavior of the circular encoder.

The equations above describe the operation of the model, but they do not suffice in convincing that it is promising for policy search, let alone suitable for regression. The NTRN was underpinned by the Nadaraya-Watson kernel regression, but the same doesn't hold for the new model. Instead, it leverages the idea of functional gradient descent in kernel machines. In the following subsections, the connection between the proposed n-tuple system and this form of learning with kernels is established.

3.1. Connection to kernel methods

As the update rule (27) and output function (29) are considerably different from their counterparts in the NTRN, it is not obvious that the new network still implements a kernel machine. However, it is straightforward to show that it does. Once again, the proof revolves around the idea of tuple distance, ρ, a measure of similarity between patterns in terms of the intersection of their tuple sets.

First, the content of a memory position at any point in time may be stated in terms of all model updates up to that time (30). $M_t^{[i]}$, here, serves as an indicator variable denoting whether $\mathbf{x}$ addresses the same memory position as $\mathbf{x}_t$ in neuron i (31). $\tau^{[i]}(\mathbf{x})$ denotes the tuple formed by input $\mathbf{x}$ for neuron i.

$$v_t^{[i]}(\mathbf{x}) = \sum_{j=1}^{t} \eta_j \delta_j M_j^{[i]}(\mathbf{x}) \tag{30}$$

$$M_j^{[i]}(\mathbf{x}) = \begin{cases} 1 & \text{if } \tau^{[i]}(\mathbf{x}) = \tau^{[i]}(\mathbf{x}_j) \\ 0 & \text{otherwise} \end{cases} \tag{31}$$

As in the NTRN, a relationship may be established between the summation of the indicator variables across all neurons and the tuple distance, and this distance, in turn, may be seen in terms of a similarity function, that is, a *kernel* (32).

$$\begin{aligned} \textstyle\sum_{i=1}^{N} M_t^{[i]}(\mathbf{x}) &= N - \rho(\mathbf{x}, \mathbf{x}_t) \\ &= N\left(1 - \tfrac{\rho(\mathbf{x}, \mathbf{x}_t)}{N}\right) \\ &= Nk(\mathbf{x}, \mathbf{x}_t), \quad k(\mathbf{x}, \mathbf{x}_t) = \left(1 - \tfrac{\rho(\mathbf{x}, \mathbf{x}_t)}{N}\right) \end{aligned} \tag{32}$$

Finally, (30) and (32) can be replaced into the output function (29). The result (33) demonstrates that the network does indeed implement a kernel machine: its output can be written as a summation of basis functions, positioned over the data points and with weights $\eta_j \delta_j, j = 1, \ldots, t$ learned from said data.

$$\begin{aligned} f_t(\mathbf{x}) &= \tfrac{1}{N} \sum_{i=1}^{N} v_t^{[i]}(\mathbf{x}) \\ &= \tfrac{1}{N} \sum_{i=1}^{N} \sum_{j=1}^{t} \eta_j \delta_j M_j^{[i]}(\mathbf{x}) \quad \text{Using (30)} \\ &= \tfrac{1}{N} \sum_{j=1}^{t} \eta_j \delta_j \sum_{i=1}^{N} M_j^{[i]}(\mathbf{x}) \\ &= \tfrac{1}{N} \sum_{j=1}^{t} \eta_j \delta_j N k(\mathbf{x}, \mathbf{x}_t) \quad \text{Using (32)} \\ &= \sum_{j=1}^{t} \eta_j \delta_j k(\mathbf{x}, \mathbf{x}_j) \end{aligned} \tag{33}$$

3.2. Implicit functional gradient descent

Proving that the network amounts to the summation of basis functions, as done above, is only the first step in understanding its connection to kernel methods. The next one concerns how the basis functions weights are learned and, more specifically, how that ties into functional gradient descent. This can be seen by noticing that a recurrence relation, in the form of (34), can be stated about the content of a memory position in this model.

$$v_t^{[i]}(\mathbf{x}) = v_{t-1}^{[i]}(\mathbf{x}) + \eta_t \delta_t M_t^{[i]}(\mathbf{x})$$

The key idea is that, according to the update rule, for each time step the memory positions that are addressed by the data point have their contents incremented by $\eta_t \delta_t$. The other memory positions, on the other hand, remain unchanged. Both possibilities can be unified, by means of the indicator variable $M_t^{[i]}$, in a single equation that relates the value of an arbitrary position at a given moment with its value in the previous step (34).

By replacing (34) into (29), a recurrence relation for the output function of the model can also be established (36).

$$\begin{aligned} f_t(\mathbf{x}) &= \tfrac{1}{N} \sum_{i=1}^{N} v_t^{[i]}(\mathbf{x}) \\ &= \tfrac{1}{N} \sum_{i=1}^{N} \left[v_{t-1}^{[i]}(\mathbf{x}) + \eta_t \delta_t M_t^{[i]}(\mathbf{x}) \right] && \text{Using (34)} \\ &= \tfrac{1}{N} \sum_{i=1}^{N} v_{t-1}^{[i]}(\mathbf{x}) + \tfrac{\eta_t \delta_t}{N} \sum_{i=1}^{N} M_t^{[i]}(\mathbf{x}) \\ &= f_{t-1}(\mathbf{x}) + \tfrac{\eta_t \delta_t}{N} \sum_{i=1}^{N} M_t^{[i]}(\mathbf{x}) && \text{Using (29)} \\ &= f_{t-1}(\mathbf{x}) + \eta_t \delta_t k(\mathbf{x}_t, \mathbf{x}) && \text{Using (32)} \\ &= f_{t-1}(\mathbf{x}) + \\ &\quad \eta_t \partial_{f_{t-1}(\mathbf{x}_t)} c(\mathbf{x}_t, y_t, f_{t-1}(\mathbf{x}_t)) k(\mathbf{x}_t, \mathbf{x}) && \text{Using (28)} \\ &= f_{t-1}(\mathbf{x}) + \\ &\quad \eta_t \partial_{f_{t-1}(\mathbf{x}_t)} c(\mathbf{x}_t, y_t, f_{t-1}(\mathbf{x}_t)) \langle k(\mathbf{x}_t, \cdot), k(\cdot, \mathbf{x}) \rangle && \text{Using (7)} \\ &= f_{t-1}(\mathbf{x}) + \\ &\quad \eta_t \langle \partial_{f_{t-1}(\mathbf{x}_t)} c(\mathbf{x}_t, y_t, f_{t-1}(\mathbf{x}_t)) k(\mathbf{x}_t, \cdot), k(\cdot, \mathbf{x}) \rangle && \text{Using (5)} \end{aligned} \tag{35}$$

$$f_t(\mathbf{x}) = f_{t-1}(\mathbf{x}) + \eta_t \langle \nabla_f \mathscr{R}[f_{t-1}, \mathbf{x}_t, y_t], k(\cdot, \mathbf{x}) \rangle \tag{36}$$

What is noteworthy about this relationship is that it takes the form of a functional gradient descent update step, with $\mathscr{R}[f_t, \mathbf{x}_t, y_t] = c(\mathbf{x}_t, y_t, f_t(\mathbf{x}_t))$ being the differentiable risk functional that guides the optimization. With this result and the one in the previous section, the operation of the proposed n-tuple network is made clear: the system implicitly implements a kernel machine that learns according to functional gradient descent.

3.3. Positive-definiteness of the tuple distance kernel

In the last two sections, the relation between the proposed n-tuple system and online learning with kernels using functional gradient descent was elaborated. However, in doing so one result was assumed to hold without previous comment. From step (35)–(36) the gradient of the evaluation functional $\nabla_f E_{\mathbf{x}}[f_t] = k(\mathbf{x}, \cdot)$ was used. This result, though, hinges on the kernel being positive-definite and, consequently, exhibiting the reproducing property.

One central result in kernel theory is that kernels are positive-definite if, and only if, they can be represented as inner products in a feature space. One possible way to prove that the tuple distance kernel is positive-definite, therefore, is to come up with a feature map for it. Before doing so, it is important to notice that the kernel can be stated in a slightly different manner.

$$\begin{aligned} k(\mathbf{w}, \mathbf{x}) &= 1 - \tfrac{\rho(\mathbf{w}, \mathbf{x})}{N} \\ &= \tfrac{1}{N}(N - \rho(\mathbf{w}, \mathbf{x})) \\ &= \tfrac{1}{N} \sum_{i=1}^{N} [\![\tau^{[i]}(\mathbf{w}) = \tau^{[i]}(\mathbf{x})]\!] \end{aligned} \tag{37}$$

$$k(\mathbf{w},\mathbf{x}) = \frac{1}{N}\sum_{i=1}^{N}\sum_{j=0}^{2^n-1}[\![\tau^{[i]}(\mathbf{w}) = \tau^{[i]}(\mathbf{x}) = j]\!] \tag{38}$$

The tuple distance kernel is directly proportional to the number of tuples shared between the inputs (37). One alternative way of tallying this number is counting the number of tuples between inputs that are simultaneously equal to a particular value and summing for all possible tuple values (38). This somewhat convoluted accounting is the basis for a feature map (39).

$$\begin{aligned}\phi(\mathbf{x}) &= \frac{1}{\sqrt{N}}\begin{bmatrix}\phi_1(\mathbf{x})\\ \phi_2(\mathbf{x})\\ \vdots \\ \phi_{2^n}(\mathbf{x})\end{bmatrix}, \quad \text{where}\\ \phi_i(\mathbf{x}) &= \begin{bmatrix}[\![\tau^{[1]}(\mathbf{x}) = i-1]\!]\\ [\![\tau^{[2]}(\mathbf{x}) = i-1]\!]\\ \vdots \\ [\![\tau^{[N]}(\mathbf{x}) = i-1]\!]\end{bmatrix}, \quad i = 1,\dots,2^n\end{aligned} \tag{39}$$

$\phi(\mathbf{x})$ maps into a $N2^n$ binary vector, with each position denoting whether a certain tuple of x is equal to a certain value. Every N-element block refers to one possible tuple value. Now, the connection between the map and the kernel can be verified simply by the inner product of two transforms.

$$\langle\phi(\mathbf{w}),\phi(\mathbf{x})\rangle = \frac{1}{N}\sum_{i=1}^{2^n}$$

$$\langle\phi_i(\mathbf{w}),\phi_i(\mathbf{x})\rangle \tag{40}$$

$$= \frac{1}{N}\sum_{i=1}^{2^n}\sum_{j=1}^{N}[\![\tau^{[j]}(\mathbf{w}) = \tau^{[j]}(\mathbf{x}) = i-1]\!] \tag{41}$$

$$= \frac{1}{N}\sum_{j=1}^{N}\sum_{i=1}^{2^n}[\![\tau^{[j]}(\mathbf{w}) = \tau^{[j]}(\mathbf{x}) = i-1]\!]$$

$$= \frac{1}{N}\sum_{j=1}^{N}[\![\tau^{[j]}(\mathbf{w}) = \tau^{[j]}(\mathbf{x})]\!]$$

$$= k(\mathbf{w},\mathbf{x})$$

In (40), the inner product is stated as the sum of N-element inner products. Next, it is important to notice that the inner product of binary vectors corresponds to the number of positions that have simultaneously value 1 in both vectors. As the positions are associated with logical predicates, this operation corresponds to realizing a conjunction of these predicates and counting the ones which are true (41). From this point forward, it is a simple matter to show that it corresponds to the tuple distance kernel and, in doing so, proving that the kernel is positive-definite.

4. Empirical evaluations on reinforcement learning tasks

One application domain that involves both nonstationary online learning and the optimization of particular objective functions is policy search in online RL. That is the search for a probability distribution of actions conditioned on a state that maximizes the long-term cumulative reward attained by an agent. One of the simplest ways the proposed model can be employed in policy search is to use it in the framework of policy gradient methods [24]. These are algorithms where models are updated by gradient ascent, according to the gradient of a performance measure. REINFORCE [25], one member of the family of policy gradient methods, has been previously adapted to the functional gradient setting in kernel machines [14]. In practice, as both the policy gradient theorem and the model operate under the assumption of gradient descent learning, this simply means that the choice of objective function must satisfy a certain requirement.

The objective should be chosen such that its gradient is, in expected value, proportional to the gradient of the metric of performance in policy gradient: the value of the initial state. One risk functional that could be adopted is given by (42), where A_t is the advantage function, defined as the discounted sum of rewards from a given time steps onwards minus the value of the state in the current time step, $V(S_t)$ (43) (the value function is also approximated using the proposed learning model). $\gamma \in [0,1]$ is the discount rate, used to modulate the emphasis given to near-term rewards. The negative sign in (42) is necessary because the n-tuple network minimizes its objective, while what is sought in policy gradient is the maximization of the initial state's value.

$$\mathscr{R}[f_t,S_t,A_t,G_t] = -A_t \ln \pi_{f_t}(A_t|S_t) \tag{42}$$

$$A_t = \sum_{k=0}^{T-t-1}\gamma^k R_{t+k+1} - V(S_t) \tag{43}$$

In this work, the policy gradient method used was Proximal Policy Optimization (PPO) [26]. PPO is a popular policy gradient method that employs a surrogate objective function to (42) that is able to rapidly improve performance while avoiding performance collapse, a phenomenon common in previous methods, such as REINFORCE. The method alternates between environment interaction and policy optimization. During environment interaction, a predetermined number of timesteps are collected in a horizon. During policy optimization, the horizon is broken into minibatches, and the model is updated for a number of epochs. The implementation used was the one from RLlib [27].

To represent a policy, the n-tuple system is used to parameterize a probability distribution, chosen according to the action space of the task at hand. As actions are sampled from the parameterized distribution, its support must be the same, or at least be close to, the action space. Three common types of actions spaces are binary, discrete and continuous ones. Suitable distributions for these scenarios are Bernoulli, via a sigmoid function (44), Categorical, using the softmax function (45) ($\hat{\mathbf{e}}_i$ denotes the unit vector in the ith direction), and a gaussian distribution (46), where the covariance matrix is diagonal. Part of the output of the network determines the mean, while the other part determines the logarithm of the diagonal of the covariance: $\mu = f_\mu(s), \Sigma = \mathrm{diag}(\exp(f_\Sigma(s)))$, with the full output being $f(s) = [f_\mu(s), f_\Sigma(s)]^\intercal$

$$\pi_f^{binary}(a|s) = \frac{1}{1+e^{sign(a)f(s)}}, \quad a \in \{-1,1\} \tag{44}$$

$$\pi_f^{discrete}(a|s) = \frac{exp(f(s)^\intercal \hat{\mathbf{e}}_a)}{\sum_{i=1}^{l} exp(f(s)^\intercal \hat{\mathbf{e}}_i)}, \quad a \in \{1,\dots,l\} \tag{45}$$

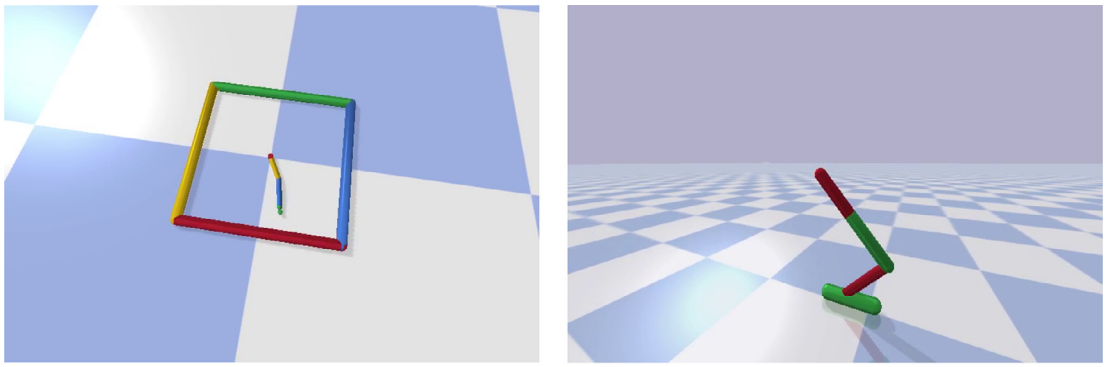
$$\pi_f^{continuous}(a|s) = \frac{1}{2}exp\Big\{-\frac{1}{2}(f_\mu(s)-a)^\intercal\Sigma^{-1}(f_\mu(s)-a)\Big\}, \quad a \in \mathbb{R}^d \tag{46}$$

4.1. Benchmark tasks and experimental considerations

Reinforcement learning algorithms are commonly tested using simulated control problems. The advantages of using simulators instead of learning directly in physical systems are numerous. It allows for simpler troubleshooting of algorithms, by separating failures stemming from the control scheme itself from other real-world sources, such as mechanical or electronic ones. Further, as learning often demands a great number of samples, interactions

(a) (b)

Fig. 5. The Cartpole (a) and Lunar Lander (b) environments.



(a) (b)

Fig. 6. The reacher (a) and hopper (b) environments.

between agent and environment can be simulated in faster than real-time.

A set of four simulated tasks from the Open AI Gym [28] and Bullet [29] libraries were used to assess the use of the *n*-tuple system in the policy search setting. These were chosen to represent different kinds of control tasks and to showcase the flexibility of the network. The simulations differ mainly in terms of their action spaces and the complexity of their dynamics.

As a representative of a task with a binary action space, cartpole (CartPole-v1) was used (Fig. 5a), consisting of a cart on a horizontal track and with a pole mounted on top of it. The pole is not rigidly fixed to the cart's frame, but held by a rotary joint. Therefore, it sways with the cart's movement and can topple towards the ground. The goal is to maintain it upright for as long as possible while avoiding the edges of the environment. The only actions available are applying a force on the cart directed left or apply it directed right. As no direct control of the angle of the pole is possible, it must be balanced through the cart's motion.

The lunar lander (LunarLander-v2) (Fig. 5b) represents a slightly more complex task than cartpole, with a set of four discrete actions. It consists of a rocket that is initially off-ground, flying over a jagged surface. As in cartpole, the vehicle resides in a bidimensional plane, but differs in its freedom of movement: through its thrusters, the rocket is able to rotate and move both horizontally and vertically. The goal is to safely descent the rocket on a landing pad. At each time step, the action consists of selecting which of the three thrusters is active or if none at all.

For continuous action spaces, two tasks were used: reacher (ReacherBulletEnv-v0) (Fig. 6a) and hopper (HopperBulletEnv-v0) (Fig. 6b). The first consists of a planar robot arm with two joints that must reach for a goal position selected at random at the start of each episode. The second task has a simplified robot with three rotary joints that is rewarded by moving forward. Given the structure of the robot, it can only do so by hopping. Both of these tasks have as actions vectors of the torques applied to each of their joints.

Table 1
Invariant hyperparameters for the encoding comparison.

Hyperparameter	Cartpole	Lunar Lander	Reacher	Hopper
Resolution	64	16	300	32
Tuple size	8	8	10	8
Discount (γ)	1.0	1.0	0.99	0.98
Learning rate (η)	3×10^{-3}	3×10^{-3}	7×10^{-7}	4×10^{-4}
Horizon	4000	2000	1000	1000
Minibatch size	128	128	128	512
Epochs	1	1	1	1

In the following sections, experiments are made using the aforementioned benchmark tasks and the resulting RL agents are compared using their average learning curves. These curves were made by training each agent 20 times, with distinct random seeds. During the course of learning of an agent, its performance was evaluated periodically by applying it to a separate evaluation environment for 5 episodes and averaging the cumulative rewards. Finally, the curves are plotted by taking the average and standard deviation of the results collected for the 20 random seeds. The hyperparameters used were selected via grid search, varying the learning rate, discount rate, number of samples in the horizon, minibatch size, number of epochs, tuple size and encoding

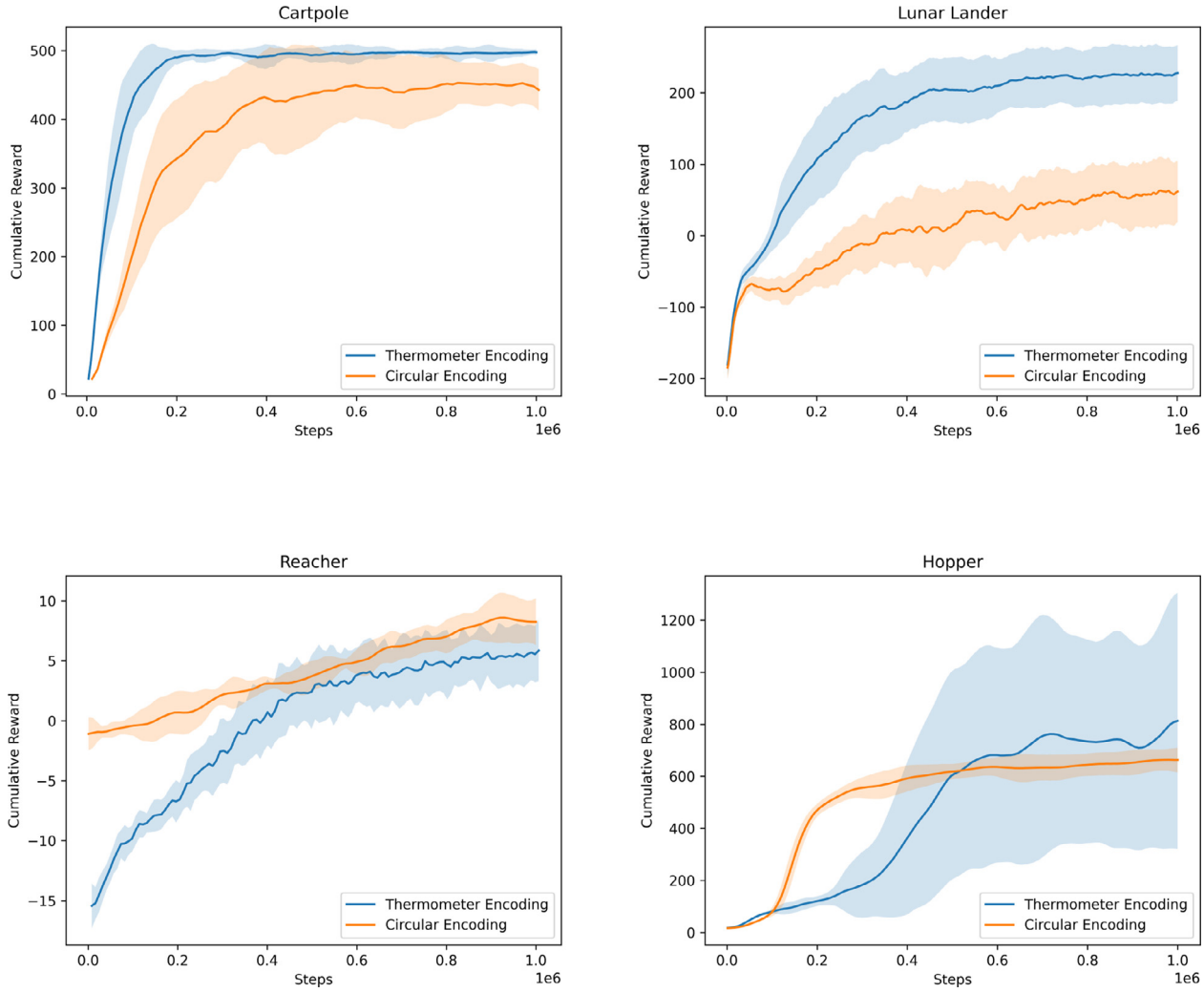


Fig. 7. Learning curves for the benchmark tasks contrasting the thermometer and circular encodings.

resolution [1] The *n*-tuple networks used in 4.2 and 4.3 have their memory positions initialized with zeros while the multilayer perceptrons have their initial weights drawn from a standard normal distribution.

4.2. Effect of the encoding scheme

As discussed in Section 2.2.3, the encoding scheme adopted, that is, the rule used to convert inputs to binary patterns, is of great importance in *n*-tuple systems, as these rules can promote the construction of different similarity relationships of the inputs by the learning model. Evidently, this learned relation should be as semantically relevant as possible. To observe the impact of the encoding scheme choice on an n-tuple-based policy search approach, networks were trained for each of the reinforcement learning tasks using both the thermometer and circular encodings. All remaining hyperparameters were held constant, with values according to Table 1.

The results can be seen in Fig. 7. While in hopper there is significant overlap between the learning curves, the same cannot be said for cartpole and lunar lander, where the thermometer encoding leads to better results, or reacher, where the circular encoding takes the advantage.

The use of the circular encoding makes the most sense in scenarios where values close to the edges of the input space should be regarded as similar and the task is one of regulation, where actions must be chosen to bring the state close to a given setpoint and keep in that way indefinitely. Reacher is one such task, where higher rewards are attained by bringing the end effector of the arm as close as possible to the goal position, and, therefore, greatly benefits from this binary representation.

4.3. Comparison with multilayer perceptrons

Given how some of the most prominent results in RL in the past few years made use of deep learning techniques [30–32], this sec-

Table 2
Hyperparameters for MLP agents.

Hyperparameter	Cartpole	Lunar Lander	Reacher	Hopper
Discount (γ)	0.99	1.0	0.999	0.98
Learning rate (η)	3×10^{-4}	2×10^{-3}	4×10^{-3}	1.2×10^{-3}
Horizon	4000	4000	8000	6000
Minibatch size	128	128	256	256
Epochs	3	1	10	2

[1] All code used for the experiments, including for hyperparameter selection, can be found in the following GitHub repository: https://github.com/rkatopodis/ntn_neurocomputing.

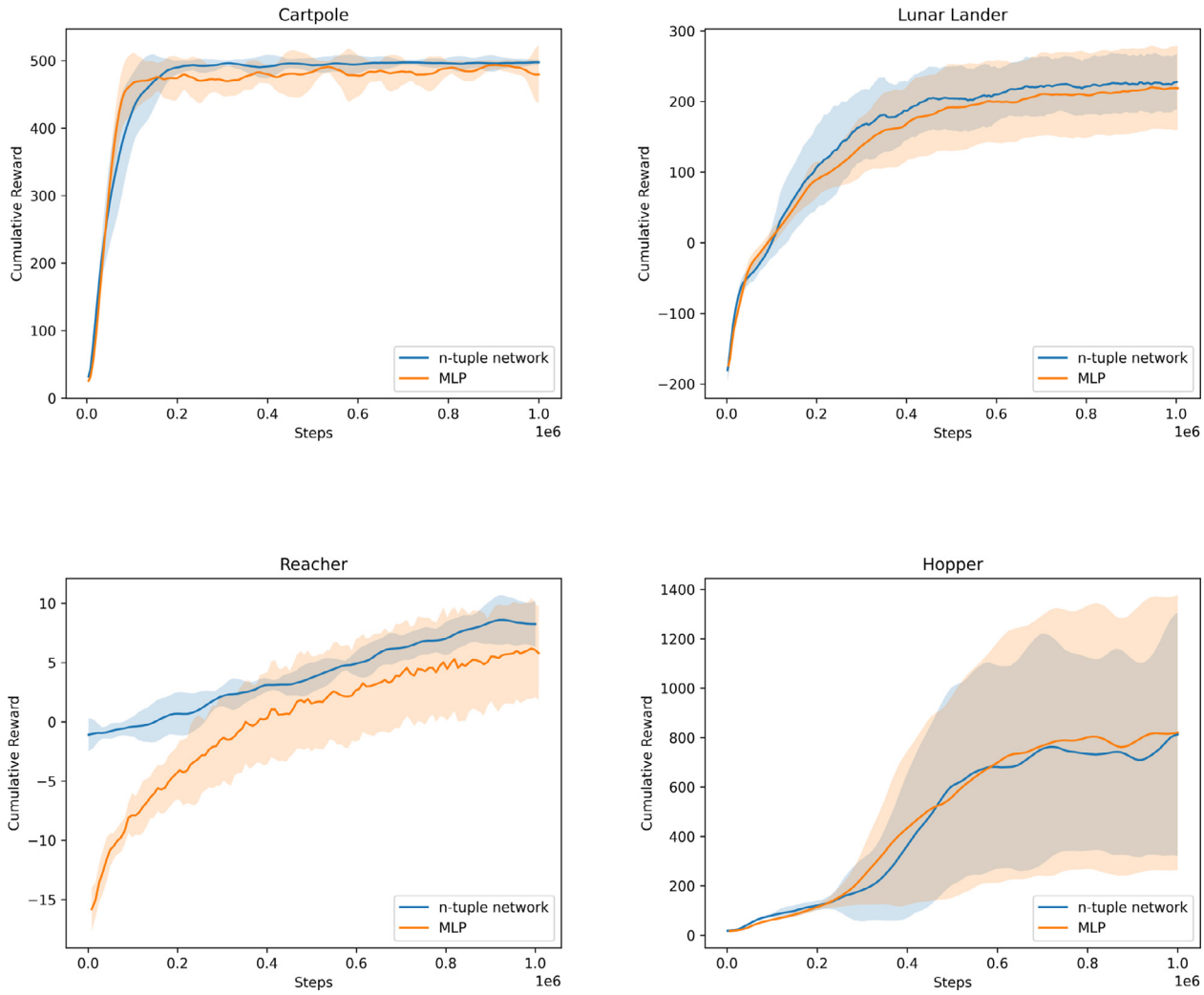


Fig. 8. Learning curves for the benchmark tasks comparing the performances attained using an *n*-tuple network and a multilayer perceptron in the context of the PPO algorithm.

tion assesses the viability of using *n*-tuple networks as policy models by comparing them against multilayer perceptrons (MLPs) in the context of the PPO algorithm and in the four benchmark tasks. All MLPs consisted of a single hidden layer with 32 units, using **tanh** activation functions. The remaining hyperparameters for the agents using MLPs can be found in Table 2. For the agents that made use of *n*-tuple nets, hyperparameters are the same as in Table 1, using the best encoding schemes for each of the tasks: thermometer encoding for cartpole, lunar lander, and hopper, and circular encoding for reacher.

The results, shown in Fig. 8, point to the use of *n*-tuple nets in policy search as a competitive option, in some situations being able to display a comparable performance to MLP-based agents, such as in the cartpole, lunar lander and hopper, and in others even surpass them, such as in reacher. Given the similar performances, one of the advantages of these *n*-tuple networks is that their operation during inference consists solely of table lookups and additions, not requiring any matrix multiplication, such as in deep learning. This characteristic makes the implementation of trained *n*-tuple nets in hardware exceptionally efficient [33]. On the other hand, one disadvantage is the need for encoding inputs as binary patterns, as it introduces hyperparameters that require tuning (the choice of encoding scheme and resolution). The process of encoding inputs may also entail a loss of information.

5. Conclusion

A new and more flexible *n*-tuple network architecture for regression has been proposed. By leveraging the idea of functional gradient descent, this model can be applied to nonstantionary online learning problems with differential objective functions, such as policy search in RL. Building upon the foundation of the NTRN, this new architecture inherits the property of inferring values in constant time. Explicit kernel methods, on the other hand, would have to resort to techniques such as truncation in order to avoid an inference time-complexity linear in the number of training samples. Finally, the network was showcased in tasks of policy search, evaluating the impact of the encoding scheme choice and comparing them to multilayer perceptrons in this domain. Ongoing work is being done in comparing the proposed approach for RL to established ones, along dimensions such as sample efficiency and stability. Future research could explore the use of adaptive learning rates and curvature information to accelerate convergence.

CRediT authorship contribution statement

Rafael F. Katopodis: Conceptualization, Methodology, Software, Writing – original draft. **Priscila M.V. Lima:** Conceptualiza-

tion, Methodology, Writing – review & editing, Supervision. **Felipe M.G. França:** Conceptualization, Methodology, Writing – review & editing, Supervision.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

Funding: This work was supported by CAPES, and CNPq, and FAPERJ.

Appendix A. Supplementary data

Supplementary data associated with this article can be found, in the online version, athttps://doi.org/10.1016/j.neucom.2022.05.114.

References

[1] H.C.C. Carneiro, C.E. Pedreira, F.M.G. França, P.M.V. Lima, A universal multilingual weightless neural network tagger via quantitative linguistics, Neural Networks 91 (2017) 85–101, https://doi.org/10.1016/j.neunet.2017.04.011.
[2] J.Y. Do, V. da Cruz Ferreira, H. Bobarshad, M. Torabzadehkashi, S. Rezaei, A. Heydarigorji, D.F.P. de Souza, B.F. Goldstein, L. Santiago, M.S. Kim, P.M.V. Lima, F.M.G. França, V.C. Alves, Cost-effective, energy-efficient, and scalable storage computing for large-scale AI applications, ACM Trans. Storage 16 (4) (2020) 21:1–21:37. doi:10.1145/3415580..
[3] M. Simões, R. Monteiro, J. Andrade, S. Mouga, F. França, G. Oliveira, P. Carvalho, M. Castelo-Branco, A novel biomarker of compensatory recruitment of face emotional imagery networks in autism spectrum disorder, Frontiers in Neuroscience 12. doi:10.3389/fnins.2018.00791..
[4] D. de O. Cardoso, J. Gama, F.M.G. França, Weightless neural networks for open set recognition, Mach. Learn. 106 (9–10) (2017) 1547–1567. doi:10.1007/s10994-017-5646-4..
[5] W.W. Bledsoe, I. Browning, Pattern recognition and reading by machine, in: F.E. Heart (Ed.), Papers presented at the 1959 eastern joint IRE-AIEE-ACM computer conference, IRE-AIEE-ACM 1959 (Eastern), Boston, Massachusetts, USA, December 1-3, 1959, ACM, 1959, pp. 225–232..
[6] A. Kolcz, N.M. Allinson, N-tuple regression network, Neural Networks 9 (5) (1996) 855–869, https://doi.org/10.1016/0893-6080(95)00116-6.
[7] L.L. Filho, L.F.R. Oliveira, A.L. Filho, G.P. Guarisa, L.M. Felix, P.M.V. Lima, F.M.G. França, Extending the weightless wisard classifier for regression, Neurocomputing 416 (2020) 280–291, https://doi.org/10.1016/j.neucom.2019.12.134.
[8] N. Cesa-Bianchi, F. Orabona, Online learning algorithms, Annu. Rev. Stat. Appl. 8 (2021) 165–190.
[9] G. Ditzler, M. Roveri, C. Alippi, R. Polikar, Learning in nonstationary environments: A survey, IEEE Comput. Intell. Mag. 10 (4) (2015) 12–25, https://doi.org/10.1109/MCI.2015.2471196.
[10] A. Shashua, Introduction to machine learning: Class Notes 67577, CoRR abs/0904.3664. arXiv:0904.3664..
[11] B. Schölkopf, A.J. Smola, Learning with Kernels: support vector machines, regularization, optimization, and beyond, Adaptive computation and machine learning series, MIT Press, 2002. URL: https://www.worldcat.org/oclc/48970254..
[12] J.A. Bagnell, D.C. Smith, Functional gradient descent (statistical techniques in robotics lecture notes) (Tech. rep.), Carnegie Mellon University, Pittsburgh, PA, 2012.
[13] J. Kivinen, A.J. Smola, R.C. Williamson, Online learning with kernels, in: T.G. Dietterich, S. Becker, Z. Ghahramani (Eds.), Advances in Neural Information Processing Systems 14 [Neural Information Processing Systems: Natural and Synthetic, NIPS 2001, December 3–8, 2001, Vancouver, British Columbia, Canada], MIT Press, 2001, pp. 785–792. URL: https://proceedings.neurips.cc/paper/2001/hash/bd5af7cd922fd2603be4ee3dc43b0b77-Abstract.html..
[14] J.A.D. Bagnell, J. Schneider, Policy search in reproducing kernel hilbert space, Tech. Rep. CMU-RI-TR-03-45, Carnegie Mellon University, Pittsburgh, PA (November 2003)..
[15] I. Aleksander, M. DeGregorio, F.M.G. França, P.M.V. Lima, H. Morton, A brief introduction to weightless neural systems, in: ESANN 2009, 17th European Symposium on Artificial Neural Networks, Bruges, Belgium, April 22–24, 2009, Proceedings, 2009. URL: https://www.elen.ucl.ac.be/Proceedings/esann/esannpdf/es2009-6.pdf..
[16] W.S. McCulloch, W. Pitts, A logical calculus of the ideas immanent in nervous activity, Bull. Math. Biophys. 5 (4) (1943) 115–133.
[17] I. Aleksander, H. Morton, An introduction to neural computing, International Thomson Computer Press, Boston, 1995.
[18] H.C.C. Carneiro, C.E. Pedreira, F.M.G. França, P.M.V. Lima, The exact VC dimension of the wisard n-tuple classifier, Neural Comput. 31 (1). doi:10.1162/neco_a_01149..
[19] I. Aleksander, W. Thomas, P. Bowden, Wisard·a radical step forward in image recognition, Sensor review..
[20] E.A. Nadaraya, On estimating regression, Theory Prob. Appl. 9 (1) (1964) 141–142.
[21] G.S. Watson, Smooth regression analysis, Sankhyā: Indian J. Stat. Ser. A (1964) 359–372.
[22] I. Aleksander, T.J. Stonham, Guide to pattern recognition using random-access memories, IEE J. Comput. Digital Tech. 2 (1) (1979) 29–40.
[23] D. de O. Cardoso, D.S. Carvalho, D.S.F. Alves, D.F.P. de Souza, H.C.C. Carneiro, C. E. Pedreira, P.M.V. Lima, F.M.G. França, Credit analysis with a clustering ram-based neural classifier, in: 22th European Symposium on Artificial Neural Networks, ESANN 2014, Bruges, Belgium, April 23–25, 2014, 2014. URL: http://www.elen.ucl.ac.be/Proceedings/esann/esannpdf/es2014-107.pdf..
[24] R.S. Sutton, D.A. McAllester, S.P. Singh, Y. Mansour, Policy gradient methods for reinforcement learning with function approximation, in: S.A. Solla, T.K. Leen, K. Müller (Eds.), Advances in Neural Information Processing Systems 12, [NIPS Conference, Denver, Colorado, USA, November 29 - December 4, 1999], The MIT Press, 1999, pp. 1057–1063.
[25] R.J. Williams, Simple statistical gradient-following algorithms for connectionist reinforcement learning, Mach. Learn. 8 (1992) 229–256, https://doi.org/10.1007/BF00992696.
[26] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, CoRR abs/1707.06347. arXiv:1707.06347. URL: http://arxiv.org/abs/1707.06347.
[27] E. Liang, R. Liaw, R. Nishihara, P. Moritz, R. Fox, K. Goldberg, J.E. Gonzalez, M.I. Jordan, I. Stoica, RLlib: Abstractions for distributed reinforcement learning, in: International Conference on Machine Learning (ICML), 2018..
[28] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, W. Zaremba, Openai gym (2016). arXiv:arXiv:1606.01540..
[29] E. Coumans, Y. Bai, Pybullet, a python module for physics simulation for games, robotics and machine learning. URL: http://pybullet.org (2016–2019)..
[30] V. Mnih, K. Kavukcuoglu, D. Silver, A.A. Rusu, J. Veness, M.G. Bellemare, A. Graves, M.A. Riedmiller, A. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, D. Hassabis, Human-level control through deep reinforcement learning, Nature 518 (7540) (2015) 529–533, https://doi.org/10.1038/nature14236.
[31] D. Silver, A. Huang, C.J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T.P. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, D. Hassabis, Mastering the game of go with deep neural networks and tree search, Nature 529 (7587) (2016) 484–489, https://doi.org/10.1038/nature16961.
[32] S. Levine, C. Finn, T. Darrell, P. Abbeel, End-to-end training of deep visuomotor policies, J. Mach. Learn. Res. 17 (2016) 39:1–39:40. URL: http://jmlr.org/papers/v17/15-522.html..
[33] Z. Susskind, A. Arora, I.D.D.S. Miranda, L.A.Q. Villon, R.F. Katopodis, L.S. de Araujo, D.L.C. Dutra, P.M.V. Lima, F.M.G. Franca, M. Breternitz, L.K. John, Weightless neural networks for efficient edge inference (2022). doi:10.48550/ARXIV.2203.01479. url:https://arxiv.org/abs/2203.01479.

Rafael F. Katopodis holds a B.Sc. in Computer Science (magna cum laude, 2019) and a M.Sc. in Systems and Computing Engineering (2022) from the Federal University of Rio de Janeiro (UFRJ), Brazil. His research interests broadly lie in the area of machine learning, with a particular focus on sequential decision-making and lighter-weight neural network architectures, such as n-tuple networks.

Priscila M.V. Lima B.Sc. in Computer Science from Federal University of Rio de Janeiro (UFRJ) (Magna cum Laude, 1982), M.Sc. in Systems Engineering and Computer Science from COPPE/UFRJ (1987), and her Ph.D. from the Department of Computing, Imperial College London, U.K. (2000). She holds the Chair of Artificial Intelligence of the Brazilian College of Advanced Studies (CBAE) of the UFRJ, Brazil, as a Professor at Tercio Pacitti Institute, and Professor at the Systems Engineering and Computer Science Program, COPPE, UFRJ, Brazil. She has research and teaching interests in artificial intelligence, artificial neural networks, computational intelligence, weightless neural networks, computational logic, distributed algorithms and other aspects of parallel and distributed computing.

Felipe M.G. França is an Invited Full Professor of Computer Science and Engineering, COPPE, Federal University of Rio de Janeiro (UFRJ), Brazil. He received his Electronics Engineer degree from UFRJ (1982), the M.Sc. in Computer Science from COPPE/UFRJ (1987), and his Ph.D. from the Department of Electrical and Electronics Engineering of the Imperial College London, U.K. (1994). He has published over 200 scientific papers and 2 granted patents. He has experience in Computer Science and Electronics Engineering, acting on the following subjects: artificial intelligence, artificial neural networks, computational intelligence, weightless neural networks, computer architecture, cryptographic circuits, dataflow computing, distributed algorithms, collective robotics, intelligent transportation systems.